

KV Cache 分層研究：文獻盤點、SOTA 的問題與下一步

日期：2026-09-30

範圍：這兩週讀了約 60 篇論文與開源系統，也在 AMD MI300X 上做了量測。這份整理目前這個領域的狀況、各篇 SOTA 的問題、我們的硬體能做到哪裡、我的觀察，以及接下來的計畫。

標註：- 〔AMD 實測〕：在 MI300X 上量到的數字。- 〔初步〕：已經跑過，但還要照記錄規範重跑一次，才會當成定論。- 〔文獻 n〕：論文或系統原文，編號對應文末參考資料。- 〔推論〕：我的判讀，還沒有實驗支持。

重點摘要

1. 這個領域做的都是 KV cache 優化，但切入的層次差很多。我把它們分成六層（第 1 節）。論文最多、最擠的是 L2「放在哪裡」，原本的 Tiara 也在這一層。
2. 在 AMD 上量到的三件事，都跟現有論文的前提不一樣：
3. KV 改成 INT4、配 vLLM 原廠的 LRU，就拿到最佳放置策略 92–104% 的好處，剩給任何聰明放置策略的空間只有 2–3.5%〔AMD 實測，初步〕。
4. 搬運速度是軟體決定的：同一張卡、同一條 PCIe，只換模型，CPU 階的有效頻寬就差 17 倍〔AMD 實測〕。
5. 能不能降精度看模型：Qwen2.5 在 FP8/INT4 下直接壞掉；其他四個模型在 129K 長度的撈針測試中，五種精度都是 100%〔AMD 實測〕。
6. 結論：「在 L2 做一個更聰明的放置策略」這條路已經很擠，我們自己的量測也顯示它剩下的空間不大。接下來打算：
7. 把這些量測整理成一篇分析型論文，主題是「什麼時候放置策略才值得做」；
8. 機制上改往 L1（精度）、L3（傳輸）、L4（命中規則）找切入點。

1. 現況：大家做的都是 KV cache 優化，可以分成六層

先打個比方：KV cache 就像模型讀過一段文字後留下的筆記。下次遇到同一段文字就直接翻筆記，不用重讀；重讀就是重算，也就是 prefill。整個領域在回答六個問題：

L5 誰在用、用多久？	← 使用方式與契約
L4 怎麼找到筆記？	← 命中規則
L3 怎麼搬筆記？	← 傳輸
L2 筆記放哪裡？	← 放置（原本的 Tiara 在這層）
L1 筆記寫多細？	← 精度／格式
L0 要存的是什麼筆記？	← 物件種類

L0：要存的是什麼。- 一般 Transformer 只要存每個 token 的 K、V，所以 vLLM、SGLang、LMCache 都只管 KV。- Qwen3-Next、Qwen3.5、Kimi-Linear 這類 hybrid 模型不一樣：有一部分層存的是一份會一直被覆寫的「狀態」（recurrent state）。它每份約數十 MB，而且只在固定位置存下來才有用〔文獻 20〕。- SGLang

有人重現過一個情況：狀態的空間用滿了，KV 明明還在、KV 池也還有 35% 空位，命中卻變成 0/8。原因是缺了狀態，KV 就接不上〔文獻 21〕。 - 現有的分層系統都還沒處理這種新物件。

L1：筆記寫多細。 - 用 BF16、FP8、INT4 存 KV：越省空間，品質損失越大。 - 代表工作：KIVI、KVTuner 做量化；AdaptCache、EvicPress 主張壓縮和放置一起決定；VeriCache、QuantSpec 用低精度當草稿、再用高精度驗證〔文獻 9、10、17、18〕。

L2：筆記放哪裡。 - 決定每段 KV 放 GPU、CPU、SSD，還是丟掉之後要用時重算。 - 代表工作：Strata／SGLang HiCache、Cake、Pensieve、Bidaw、MTDS、LMCache、Mooncake，以及 vLLM、Dynamo 內建的規則〔文獻 1–8〕。 - 這層的論文最多。

L3：怎麼搬筆記。 - 把 KV 從 CPU／SSD 搬回 GPU，或在機器之間傳。 - 論文幾乎都假設「傳輸時間 $\approx$ 資料量 $\div$ 頻寬」。

L4：怎麼找到筆記。 - vLLM、SGLang 只認「從第一個 token 開始完全相同」的前綴：中間只要一個 token 不同，後面就全部作廢。 - CacheBlend、MEPIC 讓中段內容也能重用，但會有品質損失〔文獻 12、23〕。

L5：誰在用、用多久。 - agent 多輪互動、reasoning 模型很長的思考內容、商業 API「保證存 5 分鐘」的快取契約〔文獻 25、26〕。 - 大多數論文把這一層當成固定前提。

2. SOTA 整理：每篇在解什麼、強在哪、哪裡確實做不到

「做不到」一欄只寫原文可以查到的內容。

論文／系統	層	在解什麼	最強在哪	確實做不到的地方
Strata (OSDI'26) ／ SGLang HiCache〔1〕	L2 + L3	GPU→CPU→儲存三層，用 GPU 協助搬運並調整記憶體佈局	已在 SGLang 生產環境使用；搬運做得很快	逐出只用 LRU；沒有量化；SSD 只出現在一個子實驗；認為長 context 重算「an unattractive alternative」
Cake (ICML'25)〔2〕	L2	前段重算、後段從儲存載入，兩邊並行	同時用上算力和 I/O	沒有成本模型，也沒有 fallback（作者列為未來工作）；I/O 延遲是算出來的，沒有真實儲存裝置；假設 KV 都已存好；有設定比單純載入還慢（0.80×）
Pensieve (EuroSys'25)〔3〕	L2	依「重算成本 $\div$ 閒置時間」逐出，先丟前段	最早把「越後面的 token 重算越貴」用在逐出上	沒有 SSD 階，沒有精度
Bidaw (FAST'26)〔4〕	L2	多輪對話的分層快取	有 Belady 命中率上界；trace 公開	上界是命中率，不是延遲；「多存一份中間 tensor 換計算」只對 MHA 模型划算
MTDS (C&IS'26)〔5〕	L2	每個請求決定全載、部分載或不載	量到命中很長時不載反而快	結論與 Strata 相反；只在 4 × A10、ShareGPT 上測
AdaptCache (BigMem'25)〔9〕	L1 + L2	依品質曲線決定每段 KV 的壓縮率與放置	最早主張壓縮和放置一起決定	用歷史頻率預測未來；品質曲線每個資料集只抽 10 筆；沒有最佳解上界；5 頁 workshop
EvicPress (2025)〔10〕	L1 + L2	逐出與壓縮聯合決定	聯合比單做好 2.19×	沒有最佳解上界，不知道離最好還有多遠

論文／系統	層	在解什麼	最強在哪	確實做不到的地方
LMCache [6]	L2 + L3	vLLM／SGLang 的 CPU ／磁碟卸載外掛	生產級，被廣泛 整合	標題的大倍數（如 14×）出現在 baseline 已 接近飽和的工作點
Mooncake (FAST'25) [7]	L2 + L3	以 KV 為中心的分離式 服務架構	生產級；trace 公 開	需要多節點與 RDMA
vLLM／Dynamo 內建規則 [8]	L2	LRU／ARC、至少用過 兩次才寫 SSD 等固定規 則	簡單，而且真的 在生產環境用	規則寫死，不看硬體也不看 token 位置
py-kvcache (2026) [11]	L2	依硬體量出損益平衡 點，決定存不存、載不 載	會依硬體調整	以整段前綴為單位；沒有精度
CacheBlend (EuroSys'25)／MEPIC [12、23]	L4	讓中段內容也能重用	打破「只認開 頭」的限制	有品質損失；每個請求各自修補，結果無法跨 請求共用
Marconi (MLSys'25)／ DASC (2026) [19、 20]	L0	hybrid 模型狀態的保留 與壓縮	最早處理 hybrid 模型	只在 GPU 內；沒有 CPU／SSD
VeriCache (2026)／ QuantSpec (ICML'25) [17、18]	L1 + L2	低精度 KV 當草稿，完 整 KV 驗證	greedy 解碼下， 輸出與完整 KV 一致	VeriCache 要存兩份，沒有 SSD；QuantSpec 全在 GPU 內

看完的判斷：- 沒有一篇同時決定「放哪一層、用幾個 bit、要不要重算」。- 在放置系統裡，也沒看到有人在 token（位置）層級預測「這段該存、該載，還是該重算」。- 最接近的是 Cake，但它是讀取時依算力和 I/O 速度算出一個切點的演算法，不做預測，而且假設 KV 已經全部存好。- Pensieve 以 chunk 為單位逐出。- 單一請求內挑 token 讀的工作（Quest、ShadowKV）很多，但它們不處理跨請求的放置。- 這個判斷只相對於我讀過的論文，動手前還會再查一次。

3. 大家怎麼做評測：baseline 與資料集

我統計了 35 篇論文的評測設定：

- baseline 幾乎不重疊。
- 32 篇用了 64 種不同的 baseline，其中 41 種只出現一次。
- 任兩篇之間，72.6% 完全沒有共同的比較對象。
- 最常見的共同對手只有 LRU 和原版 vLLM，兩個都是最低標準。
- 我們目前只跟 LRU／ARC 比，是因為 vLLM 內建這兩個，不是文獻慣例：ARC 在這 35 篇中沒有被任何一篇當成 baseline。
- 請求到達的方式多半是假的：用真實時間戳的只有 2 篇，用 Poisson 的 10 篇，完全沒有到達過程的 19 篇。
- 沒有人報誤差：重點的五篇都沒報重複次數、變異數或信賴區間。
- 命中率沒有統一定義：同一份 Mooncake trace，依定義不同，命中率可以從 1.1% 算到 100%。

- 測試的 context 偏短：AdaptCache 用的 LongBench 子集平均只有 1K–11K 字，MEPIC 約 1.4–2.2K token〔23〕，LRAgent 約 1–1.5K token〔22〕。

我打算這樣比：- 分四類對手：1. LRU／ARC，當最低標準；2. 生產系統的規則（vLLM「出現夠多次才寫入」的門檻、SGLang HiCache）；3. 論文的決策規則（Pensieve、Cake、py-kvcache），沒開源的就在同一個平台上重做，Bidaw 也是這樣做的；4. 「全部改 INT4 + LRU」這個簡單做法。- 再加一個理論最佳解（oracle）當上界。- 每個數字都附信賴區間。

4. 硬體：AMD MI300X 能做什麼、不能做什麼

4.1 設備

項目	規格
GPU	1 × AMD Instinct MI300X，192 GiB HBM（主機有 8 張，容器分到 1 張）
CPU／記憶體	32 核、434.8 GiB；CPU 快取階上限 178.8 GiB
儲存	沒有本地 NVMe。只能用容器的暫存碟，底層是 RAID 陣列，不是 NVMe；網路磁碟太慢，不能用
軟體	vLLM v0.28.0（ROCm 版）

4.2 SSD 這一階目前不確定

- 容器裡看不到 NVMe，所以沒辦法用真實的 NVMe SSD 測 SSD 階。
- 暫存碟本身的循序讀取約 5,400 MiB/s，但經過 vLLM 的 SSD 路徑後，各模型的有效速度只剩 1.3–3.2 GB/s〔AMD 實測〕。
- 這個速度是由這台機器的儲存設定和軟體共同決定的，不能代表一般伺服器的 NVMe。
- 所以目前所有跟 SSD 階有關的結論都先標為不確定：包括「SSD 比重算划算嗎」「SSD 的寫入量」。要有結論，需要一台有本地 NVMe 的機器。

4.3 各種比較對象在 MI300X 上能不能重現

先解釋兩個名詞：- LRU（最近最少使用）：空間不夠時，丟掉最久沒被用到的那段 KV。- ARC：同時看「最近有沒有用」和「用過幾次」，再決定丟誰。

這兩個都是 vLLM 內建的規則。

比較對象	狀態	說明
vLLM 內建的四種設定：KV 全放 GPU、多一層 CPU 並用 LRU、多一層 CPU 並用 ARC、CPU 加 SSD 兩層	已跑	4 個模型：Llama-3.1-8B、Qwen2.5-7B-1M、Qwen3-30B-A3B、Seed-OSS-36B
全部改 INT4 + LRU	已跑（初步）	要照記錄規範重跑
理論最佳解	模擬器	假設事先知道未來所有請求；得到的空間是下界
vLLM「出現夠多次才寫入」的門檻	可以跑，還沒跑	改設定就行
Pensieve、Cake、py-kvcache 的決策規則	只能在模擬器重做	原版沒開源；要在真機上跑得改 vLLM

比較對象	狀態	說明
SGLang HiCache（Strata 的實作）	可以試，還沒測	AMD 支援九月下旬才修好
LMCache	可以試，很可能跑不起來	相依套件是 NVIDIA 專屬
Mooncake、prefill／decode 分離	不能	需要多台機器和高速網路（RDMA）
GPU 直接讀 SSD（Tutti）	不能	沒有 NVMe
能耗	不能	只有整張卡的功耗，分不出記憶體和計算

5. 我的觀察

5.1 這批論文共同優點

1. 都在「算」和「存」之間取捨。一類處理 GPU／CPU／SSD 三層放哪裡；另一類專門優化從 SSD 讀回 GPU 的路徑。收益大多落在 prefill，也就是第一個 token 出來的時間（TTFT）。
2. 手法主要有三種：
3. 依 I/O 狀況加上排程，這是作業系統的經典技巧；
4. 針對特定場景寫規則；
5. 用模型去預測。搬運多半能做成非同步，跟計算同時進行。
6. 目標場景很一致：多使用者同時使用、KV 重複度高，也就是「長 context、短輸出、高重用」，例如 RAG、文件問答、agent 共用的 context。
7. 每個機制本身都很簡單，價值在框架。常見的改進是在「命中就載入」之前多判斷一步：
8. 先比載入時間和重算時間，再決定全載、載一部分，或不載直接重算；
9. 依被重用的機率排優先權，把價值低的延後；
10. 逐出從只看最近用過（LRU），改成也看使用頻率。
11. 有幾個值得參考的思路：
12. Bidaw 討論「多存一份中間結果、換掉一部分計算」到底值不值得。
13. Strata 用 Little's Law 說明「每次搬運的量要放大」。我們量到的「每筆搬運有固定成本」可以用同一條式子解釋。
14. 另外要處理「同一段 KV 正在載入時又被請求」的情況（delayed hit）。
15. 問題都界定得很具體：會講清楚在什麼規模、什麼負載下成立。

5.2 這批論文共同短板

1. 資料不夠真實。很多用合成資料，或依假設產生負載；請求到達方式多半是 Poisson 或乾脆沒有；context 大多偏短。
2. 彼此無法比較。每篇的層次、baseline、資料集、命中率定義都不一樣。
3. 大多只做一層，其他層用預設值。而且都預設了一些系統前提，像「傳輸 $\approx$ 頻寬」「量化和放置可以疊加」。我們在 AMD 上量到這兩個前提都不成立（5.3）。
4. 品質評估偏弱。
5. 不是沒人考慮品質：AdaptCache 用品質曲線決定壓縮率，KVtuner 逐層決定精度。
6. 但樣本少、只看平均分數。實際上能不能降精度和模型有關：Qwen2.5 一降就壞，平均分數會把這種情況蓋掉〔AMD 實測；文獻 24〕。

- 7. 沒考慮 reasoning 模型的思考內容。
- 8. 模型先寫一大段思考、再回答；使用者下一輪照官方建議把思考刪掉。
- 9. 從思考那段開始，後面的 KV（包括答案）就永遠不會再命中。
- 10. 開啟 CPU 卸載時，vLLM 預設還是會把這些 KV 搬到 CPU／SSD 存起來（我讀 vLLM 原始碼確認的，還沒實際量）〔文獻 15、16〕。
- 11. 沒處理新型模型。hybrid 模型已經被 MLPerf 的 agent 測試採用，但現有的分層系統都假設只有 KV（第 1 節 L0）。

5.3 在 AMD 上量到、跟論文前提不一樣的三件事

(1) 量化會取代放置。- 全部用 BF16 時，最佳放置比最好的現成規則少花 20.66%/20.91% 的 prefill 時間，看起來空間很大。- 只把 KV 改成 INT4、配原廠 LRU，就拿到這個空間的 91.8%/103.9%，只剩 2.02%/3.52%〔AMD 實測，初步〕。- 原因：放置策略的價值來自「GPU 放不下、只好挑」。INT4 讓 KV 小了 3.77 倍，常用的 KV 大多直接放得下，就沒什麼好挑了。- 成立條件：- 模型要撐得住 INT4；- 壓力不能大到縮小後還是放不下（這個轉折點還沒量）；- 目前只看 prefill。

(2) 搬運速度是軟體決定的，而且「命中率高」和「搬得快」會互相打架。- 同一條 PCIe，vLLM 的 CPU 階有效頻寬只換模型就差 17 倍（2.27–38.28 GB/s），除了唯一的 MoE 模型（Qwen3-30B-A3B，約 66%）以外，都只用到 PCIe 的 4–8%。- 原因是 vLLM 把 KV 切成很多小塊搬，每一筆不論大小都有約 13 μs 的固定成本〔AMD 實測，初步〕。- NVIDIA GB200 上也有人量到同樣現象：描述符數量會變多，是因為 prefix 重用讓 block 散落在記憶體各處，不是因為 context 變長〔文獻 13〕。- 推論：重用越多、命中率越高，實體佈局就越碎，搬得就越慢。目前沒有論文把這件事放進成本模型〔推論，待量〕。- 要注意的前作：vLLM 官方已經寫過「把小塊合成大塊，速度提升約一個數量級」〔文獻 14〕。但在我們的 MI300X 上，多數模型還是一層一小塊，原因要先查清楚。

(3) 文獻的結論互相矛盾，矛盾來自各自的傳輸實作。- Strata 說長 context 重算不划算〔文獻 1〕；MTDS 量到命中很長時不載入反而快〔文獻 5〕。- 載入與重算哪個划算，取決於搬得多快；而搬多快是由軟體實作決定的。所以兩篇很可能只是站在不同的位置，不是誰的演算法比較好〔推論〕。

5.4 最強的方法會在哪裡做錯決定（預測，待實驗確認）

方法的前提	在哪裡不成立	預期會做錯的決定
傳輸時間 = 資料量 ÷ 頻寬（Cake 與多數論文）	同一張卡換模型，搬運速度差 17 倍	算錯「前段算、後段載」的切點
長 context 載回一定比重算划算（Strata）	搬運慢的模型與設定	應該重算卻去載入
模型撐得住低精度（全部 INT4）	Qwen2.5 族	品質崩掉
用過幾次就能預測未來（生產系統的寫入規則）	reasoning 模型的思考內容	把永遠不會再用的 KV 寫進儲存
只有 KV 需要管理	hybrid 模型	以為 KV 還在，其實接不上

做法：- 在真實 trace 上逐次記錄每個方法和理論最佳解不同的決策，算出「做錯幾次、多花多少時間」。- 畫成一張圖：橫軸是搬運成本，縱軸是記憶體壓力或模型能不能降精度，標出每個方法在哪些區域會做錯。

6. 原本的 Tiara 在 AMD 上做到哪裡

原本的想法：把 KV 放置當成「在品質限制下的線上決策」。每一段 KV 可以選放 GPU（BF16／FP8／INT4）、CPU、SSD，或丟掉重算，由學習式預測器決定。

在 AMD 上的結果：

項目	結果
理論上還有多少空間	全部 BF16 時 20.66%／20.91%（Mooncake 真實 trace，只算 prefill）〔AMD 實測〕
學習式預測器	沒有贏：比最好的現成規則（vLLM 的 CPU 加 SSD 兩層）差 9.7–47.3%〔AMD 實測〕
改成 INT4 之後	剩下的空間只有 2.02%／3.52%〔AMD 實測，初步〕
換算成端到端	prefill 只佔端到端時間的 18–22%，所以就算拿到全部空間，端到端也只快約 4%
重算與傳輸的成本比	原本說跨硬體差 32 倍；實際上在同一張 MI300X 上，只換模型或長度就在 0.72 到 11.65 之間變動。主要差異來自軟體，不是硬體〔AMD 實測〕
真機整合	策略目前只在模擬器裡跑（常數來自 MI300X 實測）。vLLM 的外掛介面只能決定「CPU 階丟誰」，丟掉重算和降精度都接不上去

結論：- 「在 L2 做一個更聰明的放置策略」這條路已經很擠，Strata、Cake、Pensieve、Bidaw 和生產系統都在做。- 我們自己的量測也顯示它剩下的空間不大：換成 INT4 以後只剩 2–3.5%，端到端約 4%。- 但這些量測本身（量化取代放置、搬運由軟體決定、文獻結論矛盾的原因）是別人沒量過、而且會推翻現有前提的現象，值得整理成論文。

Tiara 的角色也會跟著改：它不再是「更聰明的預測器」，而是「先量出這台機器、這條軟體路徑、這個模型的幾個成本常數，再決定要用 INT4、用放置策略，還是改成重算」。它不會犯 5.4 那些錯誤的理由，是這些常數是量出來的，不是寫死的假設。如果實驗顯示它在某些區域也會錯，我會照實報告。

7. 接下來要做什麼

時間	工作	目的
本週	查清楚為什麼 MI300X 上多數模型還是一層一小塊地搬	決定「搬運由軟體決定」這個發現的定位
本週	「INT4 取代放置」「每筆固定成本」兩個實驗照記錄規範重跑	讓標成「初步」的數字可以正式引用
第 2 週	量「描述符數量和命中率的關係」	驗證「命中越高、搬得越慢」
第 2 週	提高記憶體壓力，找出 INT4 從「取代」變回「互補」的轉折點	確認量化結論的適用範圍
第 2 週	量 reasoning 模型寫進 CPU 卻永遠不會被讀回的比例	驗證 5.2 第 5 點，成本低
第 3 週	用小型 hybrid 模型試一週	決定 L0 值不值得當下一篇主題
第 3 週	做 5.4 的「誰在哪裡做錯」地圖；試跑 SGLang HiCache	回答「最強方法在什麼條件下會做錯」
之後	品質測試加大樣本（每格 $n \geq 500$ ，附信賴區間）	讓「能不能降精度」的判斷有統計依據

需要老師決定的事：

1. 第一篇能不能是分析型論文？主題是「決定 KV 分層效益的是量化、傳輸實作和 prefill 佔比，不是放置策略」，再加上一個事前判斷值不值得做的準則。
2. 如果一定要新機制，候選有兩個：
3. 把 INT8 拆成兩半，前 4 bit 留 GPU、後 4 bit 放 CPU (L1 + L2) ；
4. hybrid 模型的狀態與 KV 一起分層 (L0) 。
5. SSD 階需要有本地 NVMe 的機器才能下結論，實驗室有沒有這樣的機器可以用？
6. 投稿：MLSys 2027 (10/30 截止) 時間很緊；ICPE 或 EuroMLSys 比較穩。

參考資料

1. Xie et al., Strata, OSDI'26 ; SGLang HiCache 設計文件：
https://docs.sglang.io/advanced_features/hicache_design.html
2. Jin et al., Cake, ICML'25 : <https://arxiv.org/abs/2410.03065>
3. Pensieve, EuroSys'25
4. Hu et al., Bidaw, FAST'26 : <https://www.usenix.org/system/files/fast26-hu-shipeng.pdf>
5. MTDS, Complex & Intelligent Systems, 2026 :
<https://link.springer.com/content/pdf/10.1007/s40747-025-02200-4.pdf>
6. LMCACHE : <https://arxiv.org/abs/2510.09665>
7. Mooncake, FAST'25 : <https://arxiv.org/abs/2407.00079>
8. vLLM KV offloading : https://docs.vllm.ai/en/latest/features/kv_offloading_usage/ ; NVIDIA Dynamo KVBM : <https://docs.nvidia.com/dynamo/>
9. AdaptCache : <https://arxiv.org/abs/2509.00105>
10. EvicPress : <https://arxiv.org/abs/2512.14946>
11. py-kvcache : <https://arxiv.org/abs/2609.11744>
12. CacheBlend, EuroSys'25 : <https://arxiv.org/abs/2405.16444>
13. vLLM issue #55434 (GB200 上的描述符問題) : <https://github.com/vllm-project/vllm/issues/55434>
14. vLLM blog, KV Offloading Connector (2026-01-08) : <https://vllm.ai/blog/2026-01-08-kv-offloading-connector>
15. vLLM issue #39321 (思考內容污染 prefix cache) : <https://github.com/vllm-project/vllm/issues/39321>
16. SGLang PR #23315 (不快取思考內容的選項) : <https://github.com/sgl-project/sglang/pull/23315>
17. VeriCache : <https://arxiv.org/abs/2605.17613>
18. QuantSpec, ICML'25 : <https://arxiv.org/abs/2502.10424>
19. Marconi, MLSys'25 : <https://arxiv.org/abs/2411.19379>
20. DASC : <https://arxiv.org/abs/2608.30386>
21. SGLang PR #39436 (hybrid 模型的狀態池) : <https://github.com/sgl-project/sglang/pull/39436>
22. LRAgent, ICML'26 : <https://arxiv.org/abs/2602.01053>
23. MEPIC : <https://arxiv.org/abs/2512.16822>
24. Rethinking KV Cache Compression, MLSys'25 : <https://arxiv.org/abs/2503.24000>
25. Anthropic prompt caching 文件 : <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>
26. Khailo, Keepalive Economics for Agentic Workloads : <https://arxiv.org/abs/2607.19214>